**Defining Our 3D "Meaning Space"**

In a high-dimensional model (like your 768D model), the AI learns hundreds of hidden language features automatically. For our 3D toy example, let's pre-define what our **3 axes (dimensions)** represent:

- **Dimension 1 ($X$):** Sports & Physical Activity (Range: -1 to +1)
- **Dimension 2 ($Y$):** Technology & Programming (Range: -1 to +1)
- **Dimension 3 ($Z$):** Food & Cooking (Range: -1 to +1)

**Step 2: The 5 Statements & Their Vector Conversion**

Imagine we pass these 5 sentences into our embedding model. The AI analyzes the words, understands their context, and maps them into coordinates $(X, Y, Z)$ based on our 3 dimensions:

1. **Statement A:** *"Python is a popular programming language."*
    - **Vector:** [0.1, 0.9, 0.0] *(Low sports, very high tech, neutral food)*
2. **Statement B:** *"Machine learning models process data quickly."*
    - **Vector:** [0.2, 0.95, -0.1] *(Low sports, extremely high tech, neutral food)*
3. **Statement C:** *"LeBron James scored forty points in the basketball game."*
    - **Vector:** [0.9, 0.0, 0.1] *(Extremely high sports, neutral tech, neutral food)*
4. **Statement D:** *"Always preheat your oven before baking a cake."*
    - **Vector:** [-0.1, 0.0, 0.95] *(Neutral sports, neutral tech, very high food)*
5. **Statement E:** *"Writing clean code makes software easier to maintain."*
    - **Vector:** [0.0, 0.85, 0.0] *(Neutral sports, high tech, neutral food)*

**Step 3: Processing the User Question**

Now, imagine the user submits a question to find the closest context:

- **Question:** *"How do I write a python script for data analysis?"*

The embedding model processes this question using the exact same internal weights and converts it into its own 3D vector:

- **Question Vector:** [0.15, 0.88, -0.05]

**Step 4: Finding the Closest Match (Similarity Search)**

To find which statement matches the question, the system doesn't look for matching keywords (like "Python"). Instead, it calculates the mathematical distance or angle (such as **Cosine Similarity**) between the **Question Vector** and all **5 Statement Vectors** in our 3D space.

The closer the vectors are to each other geometrically, the higher their semantic similarity score:

- **Question vs. Statement A** (Python programming): **0.99 similarity** (Very Close!)

- **Question vs. Statement B** (Machine learning data): **0.95 similarity** (Close)
- **Question vs. Statement E** (Clean code software): **0.92 similarity** (Close)
- **Question vs. Statement C** (Basketball game): **0.12 similarity** (Far apart)
- **Question vs. Statement D** (Baking a cake): **-0.05 similarity** (Opposite direction)

**Result:** The system picks **Statement A** (and closely followed by B and E) because its vector coordinates sit closest to the question's coordinates in the 3D space.

**Summary of What Happens Inside the AI:**

1. **Tokenization:** Breaking text into smaller chunks or words.
2. **Neural Network Transformation:** Passing those tokens through layers of matrix multiplications (trained on billions of sentences).
3. **Dense Vector Mapping:** Compressing that semantic meaning into a fixed array of numbers (whether 3D for our example or 768D for BERT/MiniLM).
4. **Distance Math:** Using geometric formulas to calculate which text chunks share the closest "direction" of meaning.

**Clarification of 768D**

1. "Initially starting with 768d random..."
2. "...and then while adjusting the weights and 768d for every unit..."
3. "...it is getting the meaning dynamically."